

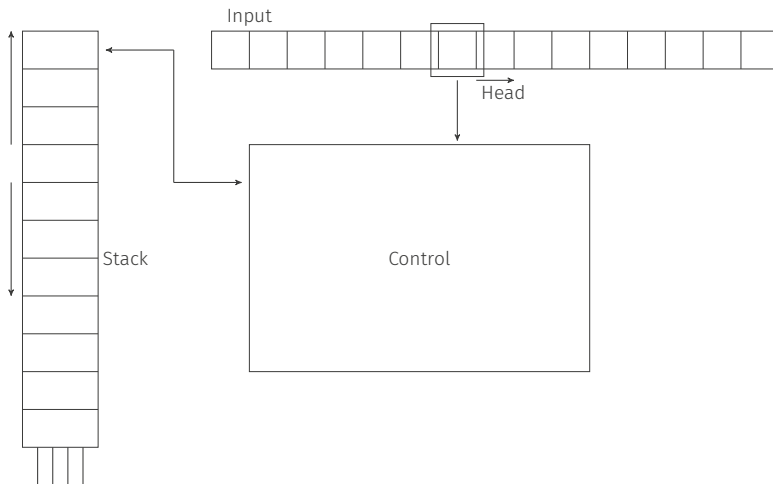
Pushdown Automata and Parser

Roland Leißa

(based on slides by S. Hack , R. Wilhelm, and M. Sagiv)

University of Göttingen

Pushdown Automata



Memory unboundedly extensible at one end,
grows (by push), shrinks (by pop), test for emptiness.

Example Automaton

- Accepted language $L = \{a^i b^i \mid i \geq 0\}$
- Context Free Grammar $S \rightarrow aSb \mid \varepsilon$
- Pushdown automaton (TOS = top of stack)

TOS	input			#
	a	b	ε	
q_0	q_0q_1	q_e	q_e	q_f
q_1	q_1q_1	q_2	q_e	q_e
q_1q_2	q_e	q_2	q_e	q_e
q_0q_2	q_e	q_e	q_e	q_f

- q_0 : initial state
- q_1 : reading a
- q_2 : reading b
- q_e : error state
- q_f : final state

Pushdown Automaton (PDA) Definition

A tuple $P = (V, Q, \rightarrow, q_0, F)$ where:

- input alphabet: V
- finite set of states (stack symbols): Q
- initial state: $q_0 \in Q$
- final states: $F \subseteq Q$
- Transition relation: $\underbrace{q_n \cdots q_1}_{n \geq 1} \xrightarrow{a} \underbrace{p_m \cdots p_1}_{m \geq 0}$, where $q_i, p_i \in Q$ and $a \in V \cup \{\varepsilon\}$
- Alternatively:
 - $\Delta \subseteq (Q^+ \times (V \cup \{\varepsilon\})) \times Q^*$
 - $\delta: (Q^+ \times (V \cup \{\varepsilon\})) \rightarrow 2^{Q^*}$, where δ is a partial function

The Language Accepted by a PDA

- PDA $P = (V, Q, \rightarrow, q_0, F)$
- For $\gamma \in Q^+$, $w \in V^*$, (γ, w) is a configuration
-

$$\text{STEP} \frac{\gamma_1 \xrightarrow{a} \gamma'_1}{(\gamma_2 \gamma_1, aw) \vdash_P (\gamma_2 \gamma'_1, w)}$$

- The language accepted by P

$$L(P) = \{w \in V^* \mid \exists q_f \in F : (q_0, w) \vdash_P^* (q_f, \epsilon)\}$$

Deterministic Pushdown Automaton

- For every $a \in V$, $\gamma_1 \xrightarrow{a} \gamma_2, \gamma'_1 \xrightarrow{a} \gamma'_2$ such that γ'_1 is a suffix of γ_1 implies:
 $\gamma_1 = \gamma'_1$ and $\gamma_2 = \gamma'_2$
- There exist no $\gamma_1 \xrightarrow{\varepsilon} \gamma_2, \gamma'_1 \xrightarrow{a} \gamma'_2$ such that $a \in V \cup \{\varepsilon\}$ and γ'_1 is a suffix of γ_1 or vice versa.

Theorem

For every context free grammar G there exists a non-deterministic pushdown automaton P such that $L(G) = L(P)$

Proof: A PDA is given which emulates the original grammar.

Context Free Items

Given context-free grammar $G = (V_N, V_T, \rightarrow, S)$

- A (context-free) **item** is a triple (A, α, β) where $A \rightarrow \alpha\beta$
- An item (A, α, β) is denoted by $[A \rightarrow \alpha.\beta]$
- Interpretation:
“In an attempt to recognize a word for A , a word for α has already been recognized”

α : **history** of the item $[A \rightarrow \alpha.\beta]$

- $[A \rightarrow \alpha.]$: A **complete** item
- IT_G : The set of items of G
- $hist([A_1 \rightarrow \alpha_1.\beta_1][A_2 \rightarrow \alpha_2.\beta_2] \dots [A_n \rightarrow \alpha_n.\beta_n]) = \alpha_1\alpha_2 \dots \alpha_n$

The Item Pushdown Automaton

- Given context-free grammar $G = (V_N, V_T, \rightarrow, S)$
- Extend grammar: Add non-term S' , and production $S' \rightarrow S$
- $P_G = (V_T, IT_G, \rightarrow, [S' \rightarrow \cdot S], \{[S' \rightarrow S \cdot]\})$

$$\text{EXPAND } \frac{Y \rightarrow \alpha}{[X \rightarrow \beta \cdot Y \gamma] \xrightarrow{\epsilon} [X \rightarrow \beta \cdot Y \gamma][Y \rightarrow \cdot \alpha]}$$

$$\text{SHIFT } \frac{}{[X \rightarrow \beta \cdot a \gamma] \xrightarrow{a} [X \rightarrow \beta a \cdot \gamma]}$$

$$\text{REDUCE } \frac{}{[X \rightarrow \beta \cdot Y \gamma][Y \rightarrow \alpha \cdot] \xrightarrow{\epsilon} [X \rightarrow \beta Y \cdot \gamma]}$$

- Source of **nondeterminism**: there may be several productions for Y (expand)

The same but different

TOS	input	new TOS	comment
$[X \rightarrow \beta.Y\gamma]$	ϵ	$[X \rightarrow \beta.Y\gamma][Y \rightarrow \alpha.]$	$Y \rightarrow \alpha$ “expand”
$[X \rightarrow \beta.a\gamma]$	a	$[X \rightarrow \beta a.\gamma]$	“shift”
$[X \rightarrow \beta.Y\gamma][Y \rightarrow \alpha.]$	ϵ	$[X \rightarrow \beta Y.\gamma]$	“reduce”

Example

$$1 \frac{}{S' \rightarrow S}$$

$$2 \frac{}{S \rightarrow \epsilon}$$

$$3 \frac{}{S \rightarrow aSb}$$

TOS	input	new TOS	comment
$[S' \rightarrow \cdot S]$	ϵ	$[S' \rightarrow \cdot S][S \rightarrow \cdot]$	$e_{1,2}$
$[S' \rightarrow \cdot S]$	ϵ	$[S' \rightarrow \cdot S][S \rightarrow \cdot aSb]$	$e_{1,3}$
$[S \rightarrow a \cdot Sb]$	ϵ	$[S \rightarrow a \cdot Sb][S \rightarrow \cdot]$	$e_{3,2}$
$[S \rightarrow a \cdot Sb]$	ϵ	$[S \rightarrow a \cdot Sb][S \rightarrow \cdot aSb]$	$e_{3,3}$
$[S \rightarrow \cdot aSb]$	a	$[S \rightarrow a \cdot Sb]$	s_{3a}
$[S \rightarrow a \cdot Sb]$	b	$[S \rightarrow aSb \cdot]$	s_{3b}
$[S' \rightarrow \cdot S][S \rightarrow \cdot]$	ϵ	$[S' \rightarrow S \cdot]$	$r_{1,2}$
$[S' \rightarrow \cdot S][S \rightarrow aSb \cdot]$	ϵ	$[S' \rightarrow S \cdot]$	$r_{1,3}$
$[S \rightarrow a \cdot Sb][S \rightarrow \cdot]$	ϵ	$[S \rightarrow aS \cdot b]$	$r_{3,1}$
$[S \rightarrow a \cdot Sb][S \rightarrow aSb \cdot]$	ϵ	$[S \rightarrow aS \cdot b]$	$r_{3,3}$

Automaton for the Expression Grammar G_0

TOS	Input	New TOS
$[S \rightarrow .E]$	ϵ	$[S \rightarrow .E][E \rightarrow .E + T]$
$[S \rightarrow .E]$	ϵ	$[S \rightarrow .E][E \rightarrow .T]$
$[E \rightarrow .E + T]$	ϵ	$[E \rightarrow .E + T][E \rightarrow .E + T]$
$[E \rightarrow .E + T]$	ϵ	$[E \rightarrow .E + T][E \rightarrow .T]$
$[F \rightarrow (.E)]$	ϵ	$[F \rightarrow (.E)][E \rightarrow .E + T]$
$[F \rightarrow (.E)]$	ϵ	$[F \rightarrow (.E)][E \rightarrow .T]$
$[E \rightarrow .T]$	ϵ	$[E \rightarrow .T][T \rightarrow .T * F]$
$[E \rightarrow .T]$	ϵ	$[E \rightarrow .T][T \rightarrow .F]$
$[T \rightarrow .T * F]$	ϵ	$[T \rightarrow .T * F][T \rightarrow .T * F]$
$[T \rightarrow .T * F]$	ϵ	$[T \rightarrow .T * F][T \rightarrow .F]$
$[E \rightarrow E + .T]$	ϵ	$[E \rightarrow E + .T][T \rightarrow .T * F]$
$[E \rightarrow E + .T]$	ϵ	$[E \rightarrow E + .T][T \rightarrow .F]$
$[T \rightarrow .F]$	ϵ	$[T \rightarrow .F][F \rightarrow .(E)]$
$[T \rightarrow .F]$	ϵ	$[T \rightarrow .F][F \rightarrow .id]$
$[T \rightarrow T * .F]$	ϵ	$[T \rightarrow T * .F][F \rightarrow .(E)]$
$[T \rightarrow T * .F]$	ϵ	$[T \rightarrow T * .F][F \rightarrow .id]$

TOS	Input	New TOS
$[F \rightarrow \cdot(E)]$	(	$[F \rightarrow \cdot(E)]$
$[F \rightarrow \cdot id]$	id	$[F \rightarrow id \cdot]$
$[F \rightarrow (E \cdot)]$	)	$[E \rightarrow (E) \cdot]$
$[E \rightarrow E \cdot + T]$	+	$[E \rightarrow E + \cdot T]$
$[T \rightarrow T \cdot * F]$	*	$[T \rightarrow T * \cdot F]$
$[T \rightarrow \cdot F][F \rightarrow id \cdot]$	ϵ	$[T \rightarrow F \cdot]$
$[T \rightarrow T * \cdot F][F \rightarrow id \cdot]$	ϵ	$[T \rightarrow T * F \cdot]$
$[T \rightarrow \cdot F][F \rightarrow (E) \cdot]$	ϵ	$[T \rightarrow F \cdot]$
$[T \rightarrow T * \cdot F][F \rightarrow (E) \cdot]$	ϵ	$[T \rightarrow T * F \cdot]$
$[T \rightarrow \cdot T * F][T \rightarrow F \cdot]$	ϵ	$[T \rightarrow T \cdot * F]$
$[E \rightarrow \cdot T][T \rightarrow F \cdot]$	ϵ	$[E \rightarrow T \cdot]$
$[E \rightarrow E + \cdot T][T \rightarrow F \cdot]$	ϵ	$[E \rightarrow E + T \cdot]$
$[E \rightarrow E + \cdot T][T \rightarrow T * F \cdot]$	ϵ	$[E \rightarrow E + T \cdot]$
$[T \rightarrow \cdot T * F][T \rightarrow T * F \cdot]$	ϵ	$[T \rightarrow T \cdot * F]$
$[E \rightarrow \cdot T][T \rightarrow T * F \cdot]$	ϵ	$[E \rightarrow T \cdot]$
$[F \rightarrow (\cdot E)][E \rightarrow T \cdot]$	ϵ	$[F \rightarrow (E \cdot)]$
$[F \rightarrow (\cdot E)][E \rightarrow E + T \cdot]$	ϵ	$[F \rightarrow (E \cdot)]$
$[E \rightarrow \cdot E + T][E \rightarrow T \cdot]$	ϵ	$[E \rightarrow E \cdot + T]$
$[E \rightarrow \cdot E + T][E \rightarrow E + T \cdot]$	ϵ	$[E \rightarrow E \cdot + T]$
$[S \rightarrow \cdot E][E \rightarrow T \cdot]$	ϵ	$[S \rightarrow E \cdot]$
$[S \rightarrow \cdot E][E \rightarrow E + T \cdot]$	ϵ	$[S \rightarrow E \cdot]$

Stack when accepting $id + id * id$:	Remaining Input
$[S \rightarrow .E]$	$id + id * id$
$[S \rightarrow .E][E \rightarrow .E + T]$	$id + id * id$
$[S \rightarrow .E][E \rightarrow .E + T][E \rightarrow .T]$	$id + id * id$
$[S \rightarrow .E][E \rightarrow .E + T][E \rightarrow .T][T \rightarrow .F]$	$id + id * id$
$[S \rightarrow .E][E \rightarrow .E + T][E \rightarrow .T][T \rightarrow .F][F \rightarrow .id]$	$id + id * id$
$[S \rightarrow .E][E \rightarrow .E + T][E \rightarrow .T][T \rightarrow .F][F \rightarrow id.]$	$+id * id$
$[S \rightarrow .E][E \rightarrow .E + T][E \rightarrow .T][T \rightarrow F.]$	$+id * id$
$[S \rightarrow .E][E \rightarrow .E + T][E \rightarrow T.]$	$+id * id$
$[S \rightarrow .E][E \rightarrow E. + T]$	$+id * id$
$[S \rightarrow .E][E \rightarrow E + .T]$	$id * id$
$[S \rightarrow .E][E \rightarrow E + .T][T \rightarrow .T * F]$	$id * id$
$[S \rightarrow .E][E \rightarrow E + .T][T \rightarrow .T * F][T \rightarrow .F]$	$id * id$
$[S \rightarrow .E][E \rightarrow E + .T][T \rightarrow .T * F][T \rightarrow .F][F \rightarrow .id]$	$id * id$
$[S \rightarrow .E][E \rightarrow E + .T][T \rightarrow .T * F][T \rightarrow .F][F \rightarrow id.]$	$*id$
$[S \rightarrow .E][E \rightarrow E + .T][T \rightarrow .T * F][T \rightarrow F.]$	$*id$
$[S \rightarrow .E][E \rightarrow E + .T][T \rightarrow T. * F]$	$*id$
$[S \rightarrow .E][E \rightarrow E + .T][T \rightarrow T * .F]$	id
$[S \rightarrow .E][E \rightarrow E + .T][T \rightarrow T * .F][F \rightarrow .id]$	id
$[S \rightarrow .E][E \rightarrow E + .T][T \rightarrow T * .F][F \rightarrow id.]$	
$[S \rightarrow .E][E \rightarrow E + .T][T \rightarrow T * F.]$	
$[S \rightarrow .E][E \rightarrow E + T.]$	
$[S \rightarrow E.]$	

Lemma

If $([S' \rightarrow .S], uv) \vdash_{P_G}^* (\rho, v)$ then $\text{hist}(\rho) \xRightarrow[G]{*} u$

Corollary: $L(P_G) \subseteq L(G)$

Lemma

Let $A \in V_N$ and $w \in V_T^*$. If $A \xRightarrow[G]{*} w$, there exists $A \rightarrow \alpha \in P$ such that for all $\rho \in IT_G^*$ and $v \in V_T^*$

$$(\rho[A \rightarrow .\alpha], wv) \vdash_{P_G}^* (\rho[A \rightarrow \alpha.], v)$$

Corollary: $L(P_G) \supseteq L(G)$

Deterministic Parsers

LL(k): Deterministic left (predictive/top-down) parsers

- Read the input from left to right
- Find leftmost derivation
- Take decisions as early as possible, i.e. on expansion
- Use k symbols look ahead to decide about expansions

LR(k): Deterministic right (bottom-up) parsers

- Read the input from left to right
- Find rightmost derivation in reverse order
- Delay decisions as long as possible, i.e. until reduction
- Use k tokens look ahead to
 - decide whether to shift or reduce
(in “shift-reduce-conflicts”)
 - decide by which rule to reduce
(in “reduce-reduce-conflicts”)